

# AGRIAI 2.0

INTELLIGENT FARMING ASSISTANT FOR GHANA

## Volume VI. Sequence Poster Pack

*Read these diagrams top to bottom. Each box is a real file or vendor.*

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Computer Science, University of Ghana

Founder, designer, and sole engineer of AgriAI

Repository: ThaddeusHackz/agriai-v2

Document date: 18 August 2026

Classification: Internal forensic architecture pack

This document explains how AgriAI was built, which source files make the AI work, and how the public and admin APIs stay reliable when a provider is missing. Every claim below is taken from the live codebase, not from marketing copy.

### How to read the posters

Time flows downward. A line of the form A -> B : message means A calls B. Alt blocks are fallbacks. These are study posters, not UML for a tool.

### Poster 1. Chat answer (the main brain)

```
Farmer
-> Chat.tsx : type question, pick Twi, mode, webSearch
-> POST /api/chat/route.ts : JSON body
  route.ts -> lib/db.ts : trackQuestion, trackMessage
  alt webSearch on
    route.ts -> lib/search.ts : searchWeb(q + " Ghana 2026")
    search.ts -> Tavily or Wikipedia or DuckDuckGo
    search.ts -> contextBlock() numbered sources
  route.ts -> settings.chat : system / expert / agent prompt
  route.ts -> lib/languages.ts : languageInstruction
  alt GEMINI_API_KEY present
    route.ts -> lib/ai.ts geminiGenerateStream
    ai.ts -> Google Gemini model waterfall
    ai.ts -> route.ts onDelta
    route.ts -> Farmer : SSE event delta { text }
  else or Gemini threw
```

```

    route.ts -> lib/cloudflare.ts cloudflareChat
    cloudflare.ts -> Llama 3.3 70B Workers AI
  else both empty
    route.ts -> lib/ai.ts localAnswer
    localAnswer -> db.knowledge then FALLBACK_ANSWERS
    route.ts -> lib/db.ts mutate : save user + assistant
    route.ts -> Farmer : SSE event done { text, demo, sources, provider }
  -> Chat.tsx : markdown, source chips, provider badge

```

## Poster 2. Sick leaf (vision)

Farmer

```

-> DiseaseDetector.tsx : pick photo
-> POST /api/vision : { image data URL, language }
  vision/route.ts : size <= 4.5MB, language whitelist
  vision/route.ts -> lib/ai.ts detectCropDisease
  alt no Gemini key
    ai.ts -> FALLBACK_DISEASE demo=true
  else
    ai.ts -> Gemini multimodal JSON prompt
    loop CHAT_MODELS until JSON parses
  vision/route.ts -> Farmer : { detected, confidence, description, treatment, demo }

```

## Poster 3. Crop Visualizer

Farmer

```

-> CropStudio.tsx : "healthy maize field at sunrise in Ghana"
-> POST /api/generate
  generate/route.ts : SAFE_SUBJECTS gate
  alt Cloudflare configured
    -> lib/cloudflare.ts Flux-1-schnell 4 steps
    -> { ok, image data URL, provider cloudflare }
  else Gemini configured
    -> lib/ai.ts geminiImage IMAGE_MODELS
    -> { ok, image, provider gemini }
  else 503 or 502 with a clear error

```

## Poster 4. Voice in and voice out

VOICE IN

Farmer mic -> Chat.tsx SpeechRecognition (primary)

```

  alt browser has no SpeechRecognition
  Chat.tsx -> POST /api/transcribe multipart audio
  transcribe/route.ts -> Gemini 2.5 Flash inline audio
  -> { text } into the chat box

```

VOICE OUT

Farmer taps speak -> POST /api/tts { text, language, speed }

```

  tts/route.ts -> ElevenLabs eleven_multilingual_v2
  -> audio/mpeg -> Chat.tsx Audio element
  alt no ELEVENLABS_API_KEY : HTTP 501

```

## Poster 5. Admin login and prompt edit

Staff

```

-> /admin/login -> POST /api/admin/login

```

```
auth.ts : rate limit IP (5 / 15 min)
bcrypt compare passwordHash
startSession 32-byte token
cookie agriai_session httpOnly SameSite=Lax
-> AdminShell AITab
-> PUT /api/admin/settings { chat.systemPrompt, model, temperature }
  getSessionUser or 401
  mutate db.settings
Next farmer chat uses the new prompt with no rebuild
```

## Poster 6. Boot and memory

```
Process start
-> instrumentation.ts
alt no data/db.json and DATABASE_URL set
  -> lib/pg-store load agriai_state
  -> lib/db.ts hydrateFromPostgres, rewrite db.json
else
  -> lib/db.ts load or defaultDB seed
    seed prices, knowledge, bcrypt admin
Request write
  -> mutate() atomic tmp rename db.json
  -> schedulePostgresSave debounce mirror
```

## Poster 7. Full provider ladder (one glance)

```
CHAT text      : Gemini stream -> Cloudflare Llama -> local knowledge
VISION photo   : Gemini JSON   -> FALLBACK_DISEASE demo card
IMAGE studio   : Cloudflare Flux -> Gemini image models -> 502
SEARCH ground  : Tavily -> Wikipedia + DuckDuckGo -> empty sources
WEATHER        : OpenWeather -> Open-Meteo -> curated city card
VOICE in       : Browser STT -> Gemini audio
VOICE out      : ElevenLabs -> disable speak
STATE          : db.json -> PostgreSQL agriai_state
```

## Why these posters matter

If you can redraw Poster 1 and Poster 7 from memory, you understand how AgriAI intelligence was made to work. The AI is a ladder of real APIs plus Ghana prompts plus a local brain, orchestrated in TypeScript by Thaddeus Nii Teiko Tagoe, sole creator of AgriAI 2.0 and of this forensic pack.